

Article

SCEditor-Web: Bridging Model-Driven Engineering and Generative AI for Smart Contract Development

Yassine Ait Hsain ^{1,*} , Naziha Laaz ²  and Samir Mbarki ¹ 

¹ Information Modeling and Communication Systems Team, EDPAGS Laboratory, Faculty of Science, Ibn Tofail University, Kenitra 14000, Morocco; s.mbarki@uit.ac.ma

² LaGeS Laboratory, Hassania School of Public Works, Casablanca 20230, Morocco; laaz.naziha@ehp.ac.ma

* Correspondence: yassine.aithsain@uit.ac.ma; Tel.: +212-610-208-630

Abstract

Smart contracts are central to blockchain ecosystems, yet their development remains technically demanding, error-prone, and tied to platform-specific programming languages. This paper introduces SCEditor-Web, a web-based modeling environment that combines model-driven engineering (MDE) with generative artificial intelligence (Gen-AI) to simplify contract design and code generation. Developers specify the structural and behavioral aspects of smart contracts through a domain-specific visual language grounded in a formal metamodel. The resulting contract model is exported as structured JSON and transformed into executable, platform-specific code using large language models (LLMs) guided by a tailored prompt engineering process. A prototype implementation was evaluated on Solidity contracts as a proof of concept, using representative use cases. Experiments with state-of-the-art LLMs assessed the generated contracts for compilability, semantic alignment with the contract model, and overall code quality. Results indicate that the visual-to-code workflow reduces manual effort, mitigates common programming errors, and supports developers with varying levels of expertise. The contributions include an abstract smart contract metamodel, a structured prompt generation pipeline, and a web-based platform that bridges high-level modeling with practical multi-language code synthesis. Together, these elements advance the integration of MDE and LLMs, demonstrating a step toward more accessible and reliable smart contract engineering.

Keywords: smart contracts; blockchain; software engineering; model-driven engineering (MDE); large language models (LLMs); generative artificial intelligence (Gen-AI); code generation



Academic Editors: Leon Sterling and Xuan Bach D. Le

Received: 30 August 2025

Revised: 28 September 2025

Accepted: 3 October 2025

Published: 7 October 2025

Citation: Ait Hsain, Y.; Laaz, N.; Mbarki, S. SCEditor-Web: Bridging Model-Driven Engineering and Generative AI for Smart Contract Development. *Information* **2025**, *16*, 870. <https://doi.org/10.3390/info16100870>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Smart contracts have revolutionized the way decentralized systems are designed and implemented, providing self-executing agreements with high reliability and transparency [1]. These programmable contracts operate on blockchain platforms and enable the automation of transactions and processes without the need for intermediaries, ensuring trust and immutability [2]. Their adoption in domains such as finance [3], supply chain [4], and healthcare [5] underscores their ability to automate trust-centric processes and enforce decentralized logic. Despite this promise, developing smart contracts remains a complex task. Developers must have deep experience in blockchain programming languages like Solidity or Vyper, and understand concepts such as gas optimization, common security vulnerabilities, and the nuances of decentralized architectures [6].

This complexity creates significant barriers to broader adoption, particularly for non-expert developers and domain specialists (e.g., in finance or supply chains) who may understand business logic but lack blockchain programming expertise. Moreover, manual smart contract coding makes them prone to errors, leading to vulnerabilities such as reentry attacks and integer overflows, which have caused financial losses in high-profile incidents [7]. Tools that raise abstraction while ensuring smart contract correctness are therefore essential. These challenges highlight the need for approaches that make smart contract development more accessible, reduce errors, and improve consistency. To address these issues, researchers and practitioners have explored methodologies such as model-driven engineering (MDE), which hides low-level technical details while supporting structured and reusable development [8,9].

MDE has been increasingly applied to smart contract development, offering a structured methodology for specifying contracts through high-level models that are later transformed into executable code [10–12]. This approach allows developers to focus on the conceptual design of contracts while supporting modularity, formal validation, and reuse [13]. Despite this, MDE alone presents several limitations. It often relies on predefined transformation rules that lack flexibility across blockchain platforms [14]. Earlier graphical approaches have shown the value of model-driven methods for smart contract development, such as BPMN-based notations [15] or UML/state-machine-based tools [16].

However, these approaches struggle to represent blockchain-specific semantics, and suffer from limited expressiveness, rigid transformation rules, and low adaptability across heterogeneous platforms. In addition, MDE-based transformations do not account for execution-level constraints, such as gas optimization and platform-specific execution models, which often require manual adjustment to ensure correctness and efficiency [17]. These limitations, combined with the absence of advanced error detection, make smart contracts generated by MDE approaches susceptible to vulnerabilities [18].

On the other hand, new methodologies for smart contract development have emerged that explore the generative capabilities of large language models (LLMs) [19–22]. By leveraging their capacity to interpret context, capture domain-specific patterns, and generalize from large-scale training in diverse code datasets, LLMs are able to automatically generate code that is readable, semantically consistent, and adaptable across multiple programming environments [23]. LLM-based generation offers a dynamic alternative to rule-based code generation by enabling the automatic generation of smart contract code that conforms to both platform-specific constraints and high-level metamodel specifications. Building on these capabilities, integrating generative AI (Gen-AI) into the MDE process has the potential to significantly improve flexibility, enable intelligent optimization, and support proactive vulnerability mitigation throughout the code generation phase.

Despite these advances, there remains a clear gap in the literature. No existing approach provides an integrated environment that bridges formal, platform-independent smart contract graphical modeling with LLM-based code generation, while also supporting reproducibility, cross-platform adaptability, and runtime-aware validation. To address this gap, we present SCEditor-Web, a web-based smart contract editor that combines the strengths of MDE, and LLM-based code generation to support key phases of the smart contract development lifecycle, particularly design and code generation. This new class of hybrid methodologies builds on previous work in model-driven engineering and AI-assisted development, combining the abstract graphical representation of MDE with the generative capabilities of LLMs.

The proposed approach was validated through representative use cases and experiments with four state-of-the-art LLMs (ChatGPT-4o, Claude 3.7 Sonnet, DeepSeek-V3, and Gemini 2.5 Pro), evaluating generated contracts for compilability, semantic fidelity, and

overall quality. The results demonstrate that coupling MDE with LLM-based synthesis produces readable, correct, and platform-specific contracts with reduced manual effort. To further illustrate its applicability, we provide a Remote Purchase case study, which showcases both the clarity of the visual syntax and the robustness of the visual-to-code pipeline. Although the current study focuses on Solidity, the metamodel and workflow are designed to be blockchain-independent, paving the way for future extensions to languages such as Rust for Solana and ink! for Polkadot.

This work builds on our previously published tool, SCEditor [24], originally developed as a desktop prototype using Eclipse Sirius. To emphasize the shift toward web-based modeling, we refer to the current system as SCEditor-Web (Project repository: <https://github.com/YassineDev91/sceditor-web> (accessed on 23 September 2025)), while maintaining the naming for continuity across our research outputs. SCEditor in this context is unrelated to similarly named editors in non-blockchain or non-modeling domains.

The remainder of this paper is structured as follows. Section 2 surveys related work, with a focus on MDE and AI-based approaches for smart contract development. Section 3 presents the overall system architecture and methodology, detailing both the web-based editor and the AI-powered code generation process. Section 4 describes the evaluation design, including metrics, tools, and model setup. Section 5 presents the SCEditor-Web prototype and its underlying metamodel, together with its visual syntax, and reports experimental results based on the evaluation metrics. Section 6 positions our approach in light of recent tools and methodologies, highlighting comparative insights and research implications. It also offers a detailed examination of the strengths and remaining limitations of the study. Finally, Section 7 concludes the paper and outlines directions for future work.

2. Related Work

This section reviews the most relevant contributions to smart contract development, focusing on two primary directions. The first centers on model-driven and visual approaches, where formal abstractions and graphical representations support the design and generation of contracts. The second focuses on AI-based methods that take advantage of large language models for automated code generation, validation, and analysis. Together, these directions reflect ongoing efforts to enhance the accessibility, reliability, and efficiency of smart contract engineering. To complement this review, we conducted a comparative analysis of the selected approaches based on predefined evaluation criteria to assess their relative strengths, weaknesses, and applicability.

2.1. Model-Driven and Visual Smart Contract Development

Model-driven engineering has gained significant traction as a methodology to simplify smart contract development and support platform independence. The research [25] proposed a high-level metamodel (HLM-SC) to model Ethereum smart contracts. Their approach enables users to model contracts using structured representations, which are then automatically transformed into Solidity code. This work demonstrates the effectiveness of MDE principles in reducing development complexity while maintaining semantic correctness.

In parallel, Jurgelaitis et al. [16] proposed a layered model-driven architecture integrating UML state machines and class diagrams to generate Solidity code through model-to-model (M2M) and model-to-text (M2T) transformations. Their evaluation confirms the viability of abstract behavior modeling for producing correct contract code. Building on this direction, Nassar et al. [18] extended the MDE landscape by introducing a computation-independent model (CIM) driven by graph grammar rules, which systematically translates into platform-independent models (PIMs) for blockchain systems. This

early-phase modeling not only enables formal verification but also supports the integration of specific requirements such as asset tracking and consensus handling.

Another relevant contribution is MUISCA [26], an MDE-based framework designed to model and generate smart contracts in Java and Go, specifically for healthcare ecosystems. This work addresses interoperability challenges across blockchain platforms. Expert evaluations confirm its usability in real-world applications, reinforcing the method's effectiveness in domain-specific smart contract development.

In [27], a domain-specific language (DSL) was introduced for smart contracts, leveraging the principles of feature modeling and metamodeling. This approach enables the formal specification of reusable contract components, thereby enhancing modularity and enabling consistent transformation into target languages such as Solidity or Vyper. Similarly, SmaC [28] proposes treating smart contracts as explicit models using a textual DSL and Xtext, enhancing maintainability and mitigating vulnerabilities.

A domain-specific application of visual modeling is presented in [29], who developed a graphical DSL for modeling business process collaborations in the construction industry. Their framework supports collaborative workflow modeling and automatically generates Solidity code, showcasing the role of domain adaptation in visual smart contract engineering.

Expanding on the domain-specific paradigm, Hamdaqa et al. [30] introduced iContractML.2.0, a platform-independent framework for modeling smart contracts. Their approach enables the generation of functional code across multiple blockchain platforms, including Ethereum and Hyperledger, thereby promoting broader interoperability.

In earlier work, Ait Hsain et al. [24] introduced SCEditor, a prototype graphical editor built on Eclipse Sirius and supported by an abstract metamodel, designed to standardize smart contract modeling across languages such as Solidity and Vyper. The tool facilitated visual contract design, streamlined platform migration, and demonstrated the potential of MDE for structuring and defining smart contract components.

Complementary to these efforts, Alzhrani et al. [31] proposed a collection of reusable business process modeling patterns designed for blockchain applications. Their work focuses on bridging the gap between requirements analysis and smart contract design by capturing recurring blockchain-specific processes and formalizing them into standardized patterns. While their approach emphasizes the specification of blockchain behaviors at the process level, it does not directly integrate code generation or support platform-independent metamodeling.

While MDE approaches offer useful abstractions, their reliance on static transformation rules often limits adaptability across evolving platforms. This limitation motivates our review of AI-driven code generation, particularly methods based on LLM-based for enabling more automated and flexible smart contract development.

2.2. AI-Based Smart Contract Development and Generation

Alongside formal modeling, the use of artificial intelligence, particularly large language models, has introduced new possibilities to automate smart contract generation and validation. Daspe et al. [32] conducted a comprehensive benchmark of LLMs in Ethereum smart contract development. Their findings indicate that model performance is highly prompt-sensitive and task-dependent, demanding domain-specific fine-tuning or structured input formats (e.g., JSON schemas) for reliable code generation. The Chat2Code system [33] enables users to specify software models through conversational interaction. Their framework generates corresponding code artifacts based on user inputs, illustrating how natural language interfaces can lower entry barriers and support non-expert users in blockchain application development. Building on this trajectory, Tong et al. [34] presented an AI-assisted method that transforms natural language descriptions of contract require-

ments into structured code representations using word segmentation and pattern extraction. Their system aims to bridge the gap between informal business logic and executable code.

Based on this trend, Gao et al. [35] introduced BPMN-LLM, an approach that leverages large language models to transform BPMN models into executable smart contracts. This work highlights the potential of visual process models for automated contract generation across various domains. However, despite the integration of LLMs, the approach remains limited by the expressiveness and structural rigidity of BPMN, which can hinder the modeling of complex contract logic or domain-specific nuances beyond conventional business workflows.

Beyond code generation, several studies have used AI for contract validation and security auditing. Kevin et al. [36] proposed SmartLLM, an AI-powered auditing tool that uses a fine-tuned LLaMA model enhanced with retrieval-augmented generation to evaluate adherence to Ethereum Request for Comments (ERC) standards. Their evaluation reported higher recall and accuracy compared to traditional static analyzers, highlighting the potential of LLMs for smart contract compliance checking.

Focusing specifically on vulnerability detection, Krichen et al. [37] proposed an AI-driven framework that applies supervised learning to code patterns in order to detect common bugs and suggest pre-emptive corrections during development. Expanding on this line of work, Mohan et al. [38] introduced a comprehensive AI-based lifecycle framework for smart contract design, generation, auditing, and deployment. Their work demonstrates that tightly coupled generative and analytical AI modules can automate the entire smart contract lifecycle, reducing development time by up to 30% and detecting more than 85% known vulnerabilities. However, the approach remains limited to Solidity on Ethereum and does not support cross-chain or multi-language code generation, underscoring the need for more blockchain-independent AI solutions.

Another relevant contribution is QuadraCode [39], an AI-powered multimodal representation framework for detecting smart contract vulnerabilities, demonstrating how AI can enhance contract security and resilience.

Although not explicitly AI-driven, the work by Sun et al. [40] aligns with the broader goals of automation and modularity. Their concept of Smart Contract-as-a-Service (SCaaS) introduces an architectural model for reusing secure and validated contract components across decentralized systems. This supports the principles of low-code engineering and AI-enhanced code reuse by promoting the adoption of pre-audited, composable building blocks.

2.3. Comparative Analysis

To consolidate the insights from the two major research directions, we conducted a comparative analysis of studied approaches based on seven key criteria relevant to smart contract modeling and generation. These include (i) the year of publication, to trace recent trends; (ii) the approach type, such as visual MDE, AI, DSLs, NLP, or LLM-based systems; (iii) the modeling inputs, ranging from high-level metamodels and UML diagrams to natural language and behavioral specifications; (iv) the underlying generation mechanism, whether rule-based (M2M/M2T) or LLM-driven (prompting or pattern extraction); (v) the target platforms and output formats, such as Solidity, Vyper, or platform-independent representations; (vi) the strengths claimed by each work, including usability, modularity, or formal guarantees; and (vii) the limitations, which often relate to platform specificity, static transformation rules, or lack of code generation. Table 1 summarizes key findings and presents a unified framework for evaluating methods and their applicability across blockchain platforms.

Table 1. Comparison of related approaches in model-driven and AI-based smart contract development.

Reference	Year	Approach Type	Modeling Inputs	Transformations & Code Generation	Targets and Outputs	Strengths	Limitations
[25]	2024	Visual MDE (HLM-SC)	High-level metamodel for Ethereum	M2M/M2T, Solidity code	Ethereum/Solidity	Reduces complexity, preserves semantics	Ethereum-specific, static rules
[16]	2022	MDE with UML	UML state machines and class diagrams	M2M/M2T, Solidity code	Ethereum/Solidity	Correct and gas-efficient code	Solidity-only, rigid transformations
[18]	2023	MDE/CIM	CIM with graph grammar rules	CIM to PIM	Multi-platform/PIM models	Formal verification, Distributed ledger technology (DLT) integration	Complex rules, no AI
[26]	2025	MDE (MUISCA)	Domain-specific models for eHealth	M2M/M2T	Multi-platform/eHealth	Interoperability, real-world validation	Domain-specific, limited scalability
[27]	2020	MDE/DSL	DSL with feature modeling	DSL To Solidity/Vyper	Multi-platform/Solidity, Vyper	Modularity, reusability	Static rules
[28]	2023	Textual DSL (SmaC, Xtext)	DSL-based smart contract models	M2M/M2T, Solidity code	Ethereum/Solidity	Maintainability, vulnerability mitigation	Platform-specific, text-heavy
[30]	2022	Platform-independent DSL (iContractML 2.0)	Platform-independent contract models	PIM to PSM, Multi-target generation	Ethereum, Hyperledger	Cross-platform generation, high mapping accuracy	Complexity in abstraction alignment
[29]	2024	Graphical DSL	DSL for B2B collaboration	Auto-generation to Solidity	Ethereum/Solidity	Domain adaptation, collaborative workflows	Domain-specific (construction)
[31]	2022	Blockchain business patterns	Reusable process models	No (specification only)	Not specified	Process standardization	No code generation
[24]	2024	Visual MDE (SCEditor)	Abstract metamodel + graphical editor	Not specified	Multi-platform	Standardizes modeling, simplifies migration, supports visual design	Desktop-bound, no AI, no code generation

Table 1. Cont.

Reference	Year	Approach Type	Modeling Inputs	Transformations and Code Generation	Targets and Outputs	Strengths	Limitations
[34]	2022	NLP (AIASCG)	Natural language documents	NL to structured code, AI-assisted word segmentation	Not specified	NL-to-code bridge	Limited for complex logic, lack of formal representations
[32]	2024	AI (LLM benchmark)	JSON/schema	LLM-based, Solidity generation	Ethereum/Solidity	Comparative LLM analysis	High prompt sensitivity
[35]	2025	BPMN + LLM (BPMN-LLM)	BPMN models	BPMN to code	Multi-platform/Solidity	Uses BPMN as LLM input	BPMN expressiveness limits
[33]	2021	Interactive NLP+ MDE (Chat2Code)	User dialogue	NL to code (Chat Mechanism)	Solidity, Hyperledger Composer and Microsoft Azure	Accessible to non-experts	Quality depends on dialogue
[36]	2025	AI auditing (SmartLLM)	Existing code	LLM-based, ERC compliance checking	Ethereum/Solidity	High detection precision	No generation (audit only)
[37]	2023	AI security (supervised)	Code	Vulnerability detection	Ethereum/Solidity	Preventive bug identification	Not multi-platform
[38]	2023	AI lifecycle framework	Models and code	Generation + audit + deployment	Ethereum/Solidity	End-to-end automation	Not multi-language
[39]	2024	AI multimodal (QuadraCode)	Code and representations	Vulnerability detection	Ethereum/Solidity	Security and resilience	No code generation
[40]	2025	Modular architecture (SCaaS)	Pre-validated components	Reuse (component-based)	Multi/component-based	Low-code, secure components	Depends on component library

From this analysis, we observe a recurring trade-off between structural rigor and generative flexibility. In the model-driven engineering category, most works utilize formal representations, such as UML, DSLs, metamodels, or BPMN, to represent smart contracts and enable rule-based code generation. The approaches proposed by Velasco et al. [25] and Jurgelaitis et al. [16] rely on formal modeling to generate Solidity code. Despite their strengths in abstraction and correctness, both remain tightly bound to the Ethereum ecosystem, limiting portability to other blockchain platforms. Ait Hsain et al. [24] present SCEditor, a graphical editor prototype for modeling structural and behavioral aspects of smart contracts. It validated the feasibility of a contract-specific metamodel and graphical modeling for smart contracts. However, SCEditor had notable limitations: it was IDE-bound and lacked transformation processes (M2M and M2T), preventing modeled contracts from being translated into executable code.

Nassar et al. [18] introduce CIM-to-PIM transformations to support early-phase modeling and cross-platform compatibility, though their reliance on complex formal rules limits accessibility. In contrast, DSL-based approaches [27–30] utilize DSLs tailored to domain-specific applications, offering modularity and reusability but remaining bounded by static rule engines. Other approaches focus on business process modeling using BPMN, such as [31], which focuses on formalizing reusable blockchain business processes, but do not address code generation. Gao et al. [35] also adopt BPMN models as a structured input to LLMs for code synthesis. However, its reliance on BPMN limits the expressiveness for detailed logic.

Overall, these approaches provide good results in formal structure and semantic clarity, but face limitations in adaptability, platform generalization, and automation.

In the AI-based category, four main groups emerge. Natural language-driven methods (e.g., [33,34]) improve accessibility by enabling users to describe or interact with smart contracts through informal, natural language interfaces. Despite this advantage, these approaches often struggle to handle complex logic structures and ensure syntactic correctness in the generated code. Structured generation approaches [32,35,39] offer structured generation and enhance semantic accuracy, but are often limited to Solidity. Auditing approaches [36–38] provide high vulnerability detection, but do not support code generation. Lastly, Sun et al. [40] introduce Smart Contract-as-a-Service (SCaaS), focusing on the reuse of pre-audited components. While not fully AI-based, it supports automation goals and aligns with low-code paradigms, promoting secure and composable contract development. In sum, natural language approaches prioritize usability, structured methods target semantic fidelity, and auditing works focus on security assurance. While all these approaches show promise, most of them lack formal modeling support and platform independence, which remain essential for broader applicability and trust in smart contract engineering. This comparative analysis highlights several persistent limitations in existing solutions, such as the lack of platform-independent modeling foundations, limited or missing integration with AI-based code generation, and insufficient support for runtime adaptability. These gaps motivated the development of SCEditor-Web, which addresses them through an adaptable modeling-to-code pipeline that combines platform-independent design with AI-assisted smart contract generation.

3. System Overview and Methodology

This section presents the architectural foundation and integrated workflow of our system, which aims to bridge high-level smart contract modeling and executable code generation through a fusion of model-driven engineering and prompt-based AI synthesis. Our methodology is based on five connected modules, as illustrated in Figure 1:

- **Abstract Modeling Layer:** It includes a platform-independent metamodel for abstract contract modeling;
- **Graphical Web-based Editor:** A web-based tool for designing structural and behavioral aspects of smart contracts;
- **AI code generator:** An AI-driven code generation engine that utilizes large language models;
- **Prompting Module:** A zero-shot prompt formulation strategy adapted for structured data;
- **Evaluation Framework:** It presents a selected dataset for cross-model evaluation of generated code.

Each module in our methodology is designed to address key limitations in traditional smart contract development, including limited abstraction, low interoperability, and rigid code generation rules. Together, these modules are intended to improve extensibility of the tooling, encourage reuse of validated artifacts, and support reproducible evaluation. In the following subsections, we describe the role of each module, their dependencies, and the trade-offs they introduce with respect to usability, interpretability, and performance.

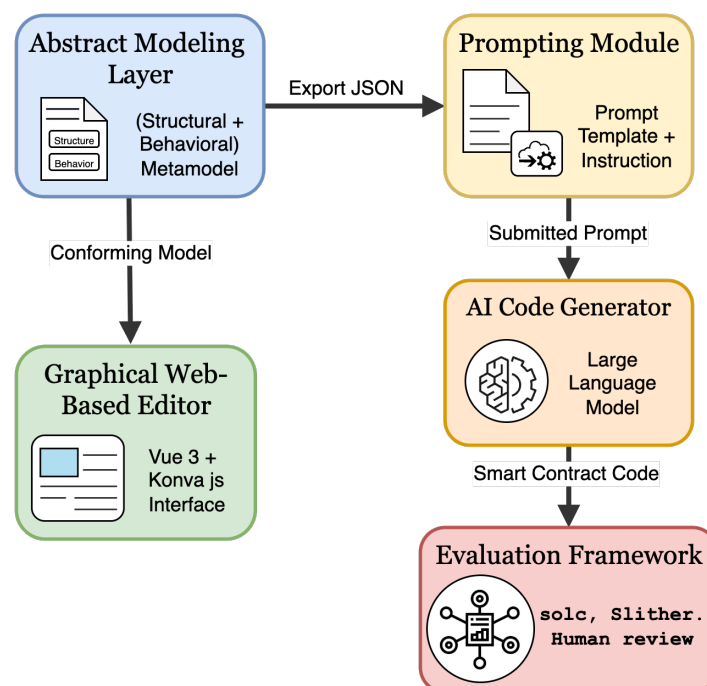


Figure 1. System architecture for model-based prompted code generation.

3.1. Model-Driven Smart Contract Design

MDE has gained prominence as a methodology for abstracting complex system development through the use of formal models that guide code generation, reduce human errors, and promote reusability [41]. In the domain of blockchain and decentralized applications, where accuracy and immutability are critical, MDE offers the potential to decouple smart contract logic from platform-specific programming constructs. This separation is part of a larger effort to enhance portability, simplify verification, and improve maintainability in smart contract development workflows.

The abstract smart contract metamodel presented in this work was designed with three principal objectives:

1. Support platform independence, enabling eventual deployment across heterogeneous blockchain systems;
2. Facilitate structured graphical modeling and formal reasoning;

3. Ensure JSON-level compatibility required for AI-driven code generation.

These objectives guided the inclusion of modeling constructs at both the structural and behavioral layers, forming a comprehensive domain-specific modeling language (DSML) tailored for smart contract abstraction and automated transformation.

This metamodel builds on our earlier version introduced in [13], where the core metaclasses and their relationships were first specified. In this work, its scope and expressiveness are extended by adding new components and semantic associations, making it more suitable for workflows that integrate AI-based code generation.

The metamodel is organized into two complementary layers. The *structural layer* captures the schema of a contract, including elements such as contracts, functions, and variables. The *behavioral layer* represents the execution logic, providing abstractions for assignments, conditionals, loops, and event emission for each declared function. The full set of metaclasses, together with their definitions and graphical representations in SCEditor-Web, is discussed in Section 5.

Unlike monolithic code blocks, the behavioral metaclasses of the metamodel (e.g., assignments, calls, loops, and conditionals) are defined as distinct, composable elements that can be hierarchically nested. This hierarchical design enables developers to model control flows and execution paths at a fine-grained level, which is critical for downstream reasoning tasks such as static analysis and dependency checking. Moreover, the explicit structure lays the groundwork for future extensions, including simulation-based validation and explainable AI techniques.

For the metamodel to be practical in AI-driven workflows, it must be both machine-readable and semantically precise. To this end, we define a deterministic serialization schema in JSON, where each model element is defined with well-structured content blocks. Listing 1 illustrates this design with an IfStatement, including its condition and body.

Listing 1. IF Statement JSON representation.

```
{
  "type": "IfStatement",
  "condition": {
    "left": "isApproved",
    "operator": "==",
    "right": "true"
  },
  "body": [
    {
      "type": "EmitStatement",
      "event": "ApprovalConfirmed"
    }
  ]
}
```

This design promotes abstraction-level stability and future-proofs the modeling approach as blockchain ecosystems evolve. Moreover, the JSON format facilitates fine-tuning and prompting of LLMs, allowing model-to-code transitions without custom parsers or AST transformations.

In summary, the metamodel serves two main purposes. First, it provides a formal modeling structure that supports user-friendly graphical modeling interfaces. Second, it functions as a structured intermediate format that is well suited to guide AI-based code generation using prompts. Consequently, it bridges the conceptual gap between

human-centered contract design and AI-driven code generation, enabling the application of generative AI within a robust, verifiable, and extensible modeling framework.

3.2. Web-Based Editor

To make the abstract smart contract metamodel operational and support real-time interaction with platform-independent models, we developed a web-based graphical editor. The editor acts as a visual modeling environment, allowing users to construct smart contracts in compliance with the metamodel described in Section 5.

Traditional modeling tools such as Eclipse Sirius [42] provide extensive capabilities but are often tied to specific IDEs, require significant training, and lack deployment flexibility. These constraints limit accessibility, particularly in domains like decentralized applications where development environments evolve rapidly. A web-based editor offers a lightweight alternative: it runs without installation, integrates naturally with modern web technologies, and provides an intuitive interface that reduces the learning curve. In addition, our approach separates smart contract modeling from IDE-dependent tooling and introduces direct, in-browser editing and execution pipelines, which allow for interactive model updates and immediate validation.

The editor is implemented using a modern frontend stack built on **Vue 3**, a progressive JavaScript framework supporting component-driven design and reactive rendering; **Konva.js**, a canvas-based library for interactive diagramming; and **Pinia**, the state management library used to synchronize model data across components. This architecture enables modular representation of metamodel entities and ensures real-time updates to model state and layout without requiring full-page reloads.

To organize smart contract design, the editor adopts a layered structure inspired by practices in visual domain-specific languages. The **MainLayer** hosts structural components such as contracts, variables, and functions, which can be spatially arranged on the canvas through clicks or drag-and-drop interactions. The **FunctionLayer** captures behavioral logic and is dynamically activated when a function node is opened, providing a clean separation between structure and execution paths. Each component is rendered through a registry mechanism, allowing new metamodel entities to be added with minimal coupling.

Modeling actions are centrally tracked in the Pinia store. Upon export, the system produces a JSON document that preserves the contract hierarchy, behavioral blocks (nested control flow), and positional metadata (e.g., x and y coordinates). This representation functions as the interface between the modeling tool and the code generation subsystem, ensuring semantic fidelity and format compatibility.

The editor further supports interaction features designed to improve usability. These include modal dialogues for component creation and confirmation steps, a sidebar drawer for previewing blockchain-specific code with syntax highlighting (`highlight.js`), and export utilities for producing both shareable images and JSON model files.

The proposed tool is available upon request on GitHub, with the current release (v0.3-alpha) supporting structural modeling, basic behavioral logic, JSON export, and prompt-driven AI code generation. The project is still under active development.

3.3. AI-Based Code Generation

Traditional MDE workflows for smart contract development rely on deterministic model-to-model and model-to-text transformations to obtain platform-specific code from abstract system models [43]. While this methodology has been proven effective in well-defined domains, it often faces rigidity when confronted with dynamic logic, language evolution, or cross-platform deployment targets. Rule-based transformations require expert configuration, are sensitive to metamodel changes, and typically demand ongoing

maintenance as either the source metamodel or the target language evolves. Moreover, these transformations tend to hard-code assumptions about control flow patterns, variable naming, and contract idioms, which limits their applicability and adaptability in real-world smart contract ecosystems.

In contrast, prompt-based generation with LLMs provides a more practical and platform-independent alternative to predefined transformation rules. Structured contract representations (e.g., JSON conforming to the metamodel) are integrated in prompts, which LLMs use to generate code related to the selected blockchain language. This approach reduces the need for complex transformations, eases the engineering workload, and adapts more naturally to the diversity of blockchain platforms.

Recent progress in LLMs has created new opportunities for translating structured inputs into natural or programming languages. Systems such as Codex [44] and CodeChain [45] show that LLMs trained on source code can perform tasks ranging from function synthesis and type inference to end-to-end logic generation. For smart contracts, where logic is contextual, stateful, and sensitive to small syntactic errors, this generative capability provides a compelling alternative to handcrafted transformation rules. Instead of relying on manually defined templates, LLMs can infer semantics directly from prompt context and structured schemas. In addition, LLMs can adapt quickly to evolving languages (e.g., new Solidity features), reducing tooling maintenance and aligning with broader developments in AI-assisted software engineering.

In our system, the JSON definition of the contract serves as the input to the code generation process. Each prompt submitted to the LLM combines the complete JSON contract model with task-specific natural language instructions, together with an additional contextual instruction that frames the generation process (see Listing 2 for the full prompt template used across all experiments).

Listing 2. LLM prompt for smart contract code generation.

You are a professional smart contract developer. Based on the JSON specification below, generate a complete [blockchain_language] smart contract.

The JSON contains:

- Structural definitions (contract, variables, structs, functions, etc.).
- Optional natural language descriptions that clarify developer intentions or behavior.

Please follow these guidelines:

- Implement all logic explicitly defined in the JSON structure.
- Use the description field (when present) to enrich the contract, infer purpose, and write readable, semantically appropriate code.
- Prioritize the description to resolve ambiguities.
- Write clean, commented, deployable code.

Here is the smart contract definition:

```
<JSON>
[contract.json]
</JSON>
```

Now generate the [blockchain_language] code. Output only the smart contract code. Do not include explanations.

The prompt is then submitted to the chosen LLM using an API or offline interface. The returned Solidity code is parsed, evaluated, and optionally reviewed for fidelity. To ensure fairness, all experiments were conducted using a consistent prompt structure and under zero-shot settings.

For evaluation, we selected four state-of-the-art LLMs representing different design philosophies and deployment contexts:

- **ChatGPT-4o** (OpenAI, 2024): A transformer-based model known for advanced reasoning and multilingual capabilities [46].
- **Claude 3.7 Sonnet** (Anthropic, 2025): Designed for instruction following and safe, high-fidelity generation [47].
- **DeepSeek-V3** (DeepSeek, 2025): An open-source alternative with high performance on structured development benchmarks [48].
- **Gemini 2.5 Pro** (Google DeepMind, 2025): Optimized for software and logic tasks, with structured document comprehension [49].

The selected LLMs were chosen based on publicly available documentation, benchmark results, and prior reports of code generation performance across open-source and proprietary contexts [44,50].

The shift from rule-based transformation to generative modeling offers several important benefits. First, it removes the need for maintaining and debugging rigid transformation rules or syntax templates. Second, it provides adaptive language support, since the same model and prompt pattern can be applied to generate code in multiple smart contract languages such as Solidity, Vyper, or Rust. Third, large language models introduce context-aware inference, allowing them to infer omitted logic, assign meaningful variable names, and reconstruct control flow from the structured input. Together, these advantages reduce engineering overhead and enable rapid prototyping across domains.

Despite their capabilities, LLMs remain probabilistic systems. Generated code may vary across runs, include non-compilable artifacts, or diverge from source model semantics. Prompt design plays a key role in minimizing such errors, and ongoing research explores reinforcement learning and structured decoding for increased reliability [51]. To systematically assess these limitations, we designed a multi-metric evaluation pipeline (Section 4) encompassing syntactic correctness, semantic alignment, and vulnerability detection. Tools such as Slither were employed to identify common vulnerability patterns and semantic mismatches. While LLMs exhibit impressive generalization, ensuring determinism, logical coherence, and contract safety remains an open challenge. This is particularly critical for operations involving fund transfers, access control, and irreversible state changes. Future work may incorporate post-generation validation pipelines, ensemble prompting, or iterative feedback loops to further improve correctness and trustworthiness.

The use of LLMs for model-to-code transformation offers a scalable, language-independent, and generalizable alternative to traditional MDE processes. Our approach demonstrates how structured JSON models can be transformed into executable contracts using prompt-based generation, with results competitive across multiple state-of-the-art models.

3.4. Prompt Strategy

Prompt engineering is a critical component of LLM-based code generation workflows, as the structure and phrasing of prompts directly influence the quality, accuracy, and security of the output [52,53]. Unlike traditional programming or rule-based transformations, prompting requires the formulation of effective natural language or structured inputs that LLMs can interpret to produce deterministic, semantically coherent code.

Figure 2 zooms into the core transformation process from JSON to code. The JSON model is interpreted by a prompt formatter, which combines structured metadata with a natural language instruction. This structured prompt is submitted to the selected LLM, which generates language-specific source code based on its learned understanding of code patterns and contract structures.

This design follows zero-shot prompting practices, where no in-context examples or task-specific fine-tuning are provided. Prior research has demonstrated that large-

scale LLMs exhibit robust reasoning and translation capabilities even under zero-shot conditions [54,55]. This decision was motivated by two key factors:

- Zero-shot prompting aligns with real-world development workflows where training datasets are sparse or unavailable.
- Fine-tuning or retrieval-augmented generation (RAG) techniques require complex infrastructure and are often limited to narrow domains or specific blockchain platforms.

To ensure fairness in evaluation, a consistent prompt template was applied across all models and contracts. Only the model name and targeted blockchain language varied per invocation. This methodological consistency allows for meaningful comparison of outputs across metrics such as syntax success, semantic fidelity, and code quality (see Section 5).

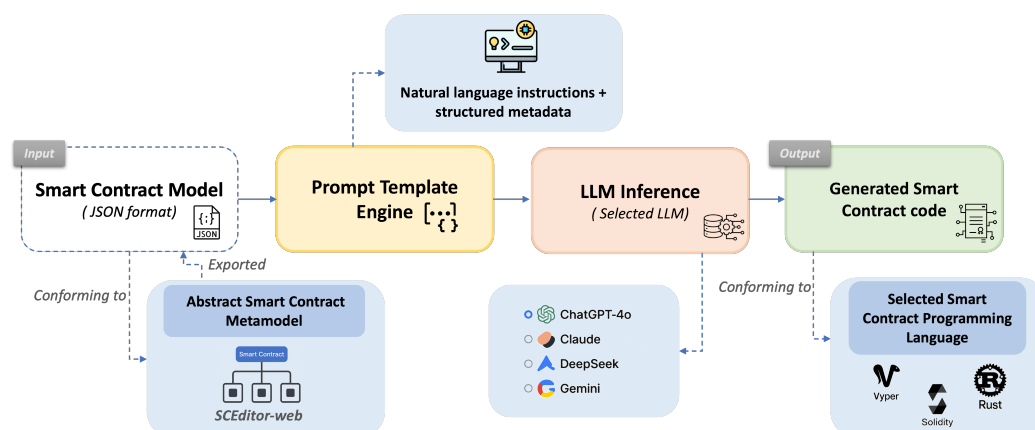


Figure 2. Process of transforming JSON-based smart contract models, exported from SCEditor-Web, into executable code through prompt-based interaction with LLMs.

3.5. Dataset: JSON–Solidity Pairs

The evaluation of generative code models, particularly in domain-specific contexts like smart contract synthesis, requires a well-structured and representative dataset. In this work, we construct a compact but functionally diverse dataset comprising platform-independent smart contract models and their corresponding generated Solidity implementations. This dataset supports benchmarking across LLMs and enables controlled, comparative analysis of syntax, semantics, and quality.

While numerous code generation benchmarks exist for general-purpose programming languages [44,54], there is a lack of datasets tailored to the smart contract domain, especially those grounded in formal model representations. Our dataset addresses this gap by focusing on use cases that are relevant, cross-domain, and structurally diverse. These include commercial transactions, digital service provisioning, and tokenized decision-making workflows.

Three distinct real-world inspired scenarios were selected for modeling:

- **Blind Auction:** Sealed bidding process with reveal and finalization phases.
- **Remote Purchase:** Conditional delivery and payment release using escrow logic.
- **Hotel Inventory Management:** Room availability, booking, and refund management.

Each scenario introduces unique challenges for generation, such as conditional flows, stateful logic, or role-based access patterns.

All corresponding visual models were designed using SCEditor-Web and exported in JSON file containing complete structural and behavioral definitions, including functions, parameters, assignments, return statements, and conditional logic.

Each model was submitted to the four evaluated LLMs under identical prompt configurations. The resulting Solidity code files were stored in an organized folder structure,

indexed by use case and model. Alongside each generated contract, we also stored the compilation results obtained from `solc`, static analysis reports produced using Slither, and manual semantic alignment scores.

Table 2 provides a concise summary of the modeled use cases and their functional characteristics.

While the dataset provides functional variety and structural clarity, its size is limited by the time required to manually validate JSON-to-code pairs. Furthermore, the current implementation focuses solely on Solidity. Future expansions may incorporate contracts in Vyper or Rust, and augment the dataset with ground truth unit tests or dynamic runtime logs.

All JSON specifications, their corresponding Solidity outputs generated by each LLM, and the full evaluation pipeline (including syntax, semantic, and quality scoring scripts) are available in the companion GitHub repository (<https://github.com/YassineDev91/smart-contract-eval> (accessed on 23 September 2025)).

Table 2. Overview of the three use case applications and their corresponding smart contract functionalities used in evaluation.

Use Case	Domain	Smart Contract Functionalities
Blind Auction	Auctions/E-Commerce	Implements sealed bid auction logic with privacy guarantees. Includes both commitment and reveal phases, time-based validation, and transfer of winning bid.
Remote Purchase	Retail/Logistics	Represents a purchase process using an escrow, where both the buyer and the seller must confirm the transaction. Encodes payment, delivery, refund conditions, and secure seller-buyer arbitration.
Hotel Inventory	Hospitality/Travel	Manages hotel room availability, booking, cancellation, and state transitions. Provides filtering, occupancy logic, and refund handling.

4. Evaluation Design

This section presents the experimental setup and metric framework used to evaluate the correctness, fidelity, and overall robustness of smart contract-generated code by the chosen LLMs from the model-driven JSON specifications. The assessment is structured around three key dimensions: syntactic validity, semantic alignment, and implementation quality.

4.1. Metrics Defined

To evaluate the correctness, alignment, and structural integrity of the AI-generated smart contracts, we defined four core metrics that collectively capture both surface-level syntax and deep behavioral fidelity. These metrics are designed to assess not only whether the code compiles, but also whether it expresses the logic faithfully and adheres to high-quality development practices.

4.1.1. Syntax Success Rate (SSR)

This metric quantifies the proportion of contracts that compiled successfully using the official Solidity compiler:

$$SSR = \frac{N_{\text{valid}}}{N_{\text{total}}} \times 100 \quad (1)$$

where N_{valid} denotes the number of contracts that compiled without errors and N_{total} is the total number of generated contracts. It provides a coarse but critical signal, as successful compilation is a prerequisite for any deployable smart contract. A 100% Syntax Success Rate implies grammatical correctness, proper structure, and baseline adherence to Solidity's type system and keywords.

4.1.2. Semantic Fidelity Score (SFS)

Each generated contract was manually evaluated against the structured JSON from which it was derived. The evaluation rubric is based on five core criteria:

- **Function Matching:** Does the generated contract include all the functions defined in the JSON specification, with appropriate naming and visibility?
- **Logic Structure:** Are control structures like `if`, `else`, `for`, and `return` statements correctly reconstructed?
- **Data Handling:** Are mappings, structs, arrays, and variables declared and accessed in a manner faithful to the original model?
- **Return Behavior:** Are output values returned in the correct format and context?
- **Control Flow Consistency:** Does the contract preserve the overall logic sequence and interaction pattern of the modeled design?

Each dimension is scored from 0 to 5, and the average yields the contract's SFS:

$$SFS = \frac{1}{5} \sum_{i=1}^5 s_i \quad \text{where } s_i \in [0, 5] \quad (2)$$

where s_i is the score for the i -th criterion. A score of 5.0 indicates perfect alignment between the generated contract and its source model, while lower values reflect partial or incorrect reconstruction.

4.1.3. Code Quality Score (CQS)

Beyond fidelity, the readability and maintainability of the generated contracts are also critical. For this reason, we defined a heuristic-based quality metric covering the following:

- **Readability:** Are indentation, line length, and code structure clear and consistent?
- **Modularity:** Are functions concise and logically separated?
- **Naming Conventions:** Do identifiers meaningfully represent their roles?
- **Gas-Aware Design:** Are Solidity-specific gas optimization patterns used appropriately?
- **Solidity Best Practices:** Is there use of `require`, proper visibility modifiers, and protection against vulnerabilities?

Each contract is scored on a 0–5 scale for each dimension, where q_k denotes the score assigned to the k -th criterion. The overall score for each contract is

$$CQS = \frac{1}{5} \sum_{k=1}^5 q_k \quad \text{where } q_k \in [0, 5] \quad (3)$$

A higher CQS indicates code that is not only correct but also maintainable, efficient, and aligned with Solidity development standards.

4.1.4. Normalized Composite Score (NCS)

To aggregate the individual metrics into a unified performance indicator per model, we define a Normalized Composite Score:

$$NCS = \frac{1}{3} \left(\frac{SSR}{100} + \frac{SFS}{5} + \frac{CQS}{5} \right) \quad (4)$$

This final score combines syntax correctness, semantic alignment, and implementation quality into a balanced single value in the range [0, 1], suitable for ranking or radar visualization.

4.2. Tools Used

To ensure objective evaluation and reproducibility, a dedicated toolchain was used for validation, analysis, and scoring. The selected tools reflect standard practices in Solidity development and smart contract verification.

4.2.1. Solidity Compiler (solc)

Syntax validation was performed using the official Solidity compiler, `solc`. Each generated contract was compiled individually, and the success or failure of this operation was used to compute the SSR. Compiler messages were logged and categorized by type to assess model consistency and error frequency.

4.2.2. Slither Static Analysis

We employed `slither`, a static analysis tool for Solidity, to scan all syntactically valid contracts for vulnerabilities and best-practice violations. The tool reports on common issues including

- Unchecked external calls;
- Uninitialized storage variables;
- Reentrancy risk;
- Inefficient gas usage;
- Missing visibility specifiers.

While Slither results were not directly included in metric scoring, they were used to qualitatively assess each model's awareness of Solidity design safety.

4.2.3. Batch Validation Pipeline

All contract files were processed through a centralized Python-based pipeline (Python 3.13). This pipeline automated compilation, Slither analysis, score logging, and report generation. It also ensured reproducibility by applying consistent processing logic across all model outputs.

4.3. Testing Setup

To assess the performance of generative models in converting structured contract models into executable Solidity code, we selected four advanced LLMs with publicly accessible APIs or interfaces:

- **ChatGPT 4o:** Accessed via OpenAI Chat API.
- **Claude 3.7 Sonnet:** Accessed via Anthropic platform.
- **DeepSeek-V3:** Accessed via DeepSeek's website.
- **Gemini 2.5 Pro:** Accessed via Google AI Studio.

Each model was evaluated in a consistent **zero-shot setting**. No examples, clarifications, or formatting instructions were provided beyond a single functional prompt.

The prompt was designed to be concise yet expressive, providing clear task framing and code generation constraints without relying on fine-tuning or in-context examples. This structure allowed us to evaluate each model's ability to generalize from the formal JSON representation while adhering to stylistic and structural expectations.

The models were prompted with three distinct contract JSON files generated through our graphical web editor. Each model thus produced three contracts, for a total of 12 *Solidity outputs* subjected to metric analysis. These contracts served as a diverse benchmark set, combining declarative modeling elements, conditional control flows, event emissions, and common Solidity data structures such as mappings, enums, and structs.

All models were evaluated under default inference settings. No temperature, max token, or stop condition tuning was performed. This ensured fairness and consistency while capturing each model's default code generation capability in realistic production-like usage.

4.4. Threats to Validity

Our evaluation, while systematic, is subject to several potential threats to validity, which we outline below:

- **Construct Validity:** The evaluation metrics used (SSR, SFS, and CQS) are designed to approximate code correctness, completeness, and quality. However, these proxies do not fully capture critical aspects such as gas efficiency, formal correctness, or deployability on real networks. Additionally, SFS relies on reference-based pattern detection, which may not cover all semantically correct alternatives.
- **Internal Validity:** The smart contract specifications used for testing were manually designed and may reflect unintentional biases or structural regularities that influence LLM outputs. Moreover, prompt formulation plays a key role in LLM performance; while we aimed for consistency across models, small changes in prompt wording can impact the generated code. We mitigated this by applying a controlled prompt generation pipeline and consistent evaluation scripts.
- **External Validity:** Our results are based on a specific set of LLMs (ChatGPT-4o, Claude 3.7 Sonnet, DeepSeek-V3 and Gemini 2.5 Pro) and target three languages (Solidity, Vyper, Rust). While these represent a broad and modern selection, findings may not generalize to other LLMs or contract platforms. Similarly, real-world smart contracts often include broader system-level interactions and external dependencies not modeled in our evaluation.
- **Conclusion Validity:** While metric-based trends were consistent across multiple LLMs and contract types, the interpretation of scores (especially SFS and CQS) can be sensitive to subjectivity in reference construction or prompt design. Our analysis focused on static evaluations and does not account for runtime behavior, gas usage, or formal verification outcomes.

Future work will address these limitations by incorporating larger and more diverse specification sets, exploring multiple prompt variants, integrating dynamic analysis, and involving domain experts in human-centered evaluations.

5. Results and Analysis

5.1. SCEditor-Web

Figure 3 shows the main interface of SCEditor-Web, which is organized into four coordinated sections that together support smart contract modeling and code generation.

The **header** (Figure 3a) provides project-level operations such as contract creation, file import/export, and user profile management. The **left panel** (Figure 3b) contains the modeling palette, which adapts dynamically depending on whether the user is working in the structural or functional view, and includes zoom controls to navigate large diagrams. At the center, the **workspace** (Figure 3c) presents the graphical contract model, where components can be placed, connected, and organized in conformance with the metamodel. The **right panel** (Figure 3d) combines element-specific editing with code generation. The upper section exposes the properties of the selected element for fine-grained adjustment,

while the lower section allows users to select a target blockchain language and trigger LLM-based code generation, which is displayed in a side drawer for immediate inspection.

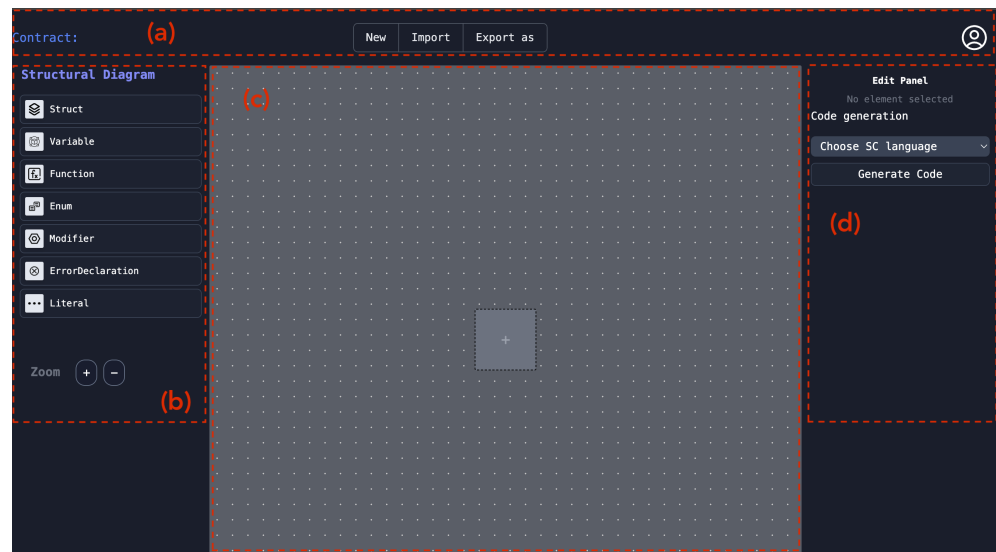


Figure 3. Main interface of SCEditor-Web in its initial state.

This layout was intentionally designed to balance usability with methodological rigor. By separating concerns (model editing, property configuration, and code generation) into distinct sections, SCEditor-Web enables a clear workflow from contract abstraction to executable code. During experimentation, the interface proved to be responsive and aligned with the metamodel, ensuring that all metaclasses could be expressed without relying on external tools. Compared with IDE-bound modeling environments, the web-based design lowers entry barriers, supports iterative development, and broadens accessibility for both expert developers and non-specialist users.

To complement the neutral interface shown in Figure 3, Table 3 summarizes the metaclasses available in SCEditor-Web. It distinguishes between structural and functional diagram elements and shows how each construct is represented in the editor palette. Some elements, such as *Literal*, appear in both layers, reflecting their role in expressing both static values and dynamic expressions.

Although the current version of the editor supports complete structural modeling and basic behavioral metaclasses (e.g., assignments, conditions), several advanced features are still under development. Planned extensions include the following:

- **AI-assisted modeling:** Real-time features such as automatic component generation, contextual code suggestions, and prompt previews within the editor interface.
- **Design-time validation:** Mechanisms for checking syntactic completeness and semantic consistency during model construction to avoid malformed exports or generation failures.
- **Advanced behavioral modeling:** Support for nested control flow, inline expressions, and chained statements.
- **Execution and simulation:** Facilities for validating contract behavior through runtime simulation and state-transition analysis.
- **Collaboration and versioning:** Built-in history tracking, multi-user editing, and integration with version control systems.
- **Interoperability:** Import capabilities for existing smart contract code and models from other DSLs, enabling reuse and migration of legacy artifacts.
- **Usability enhancements:** Undo/redo, alignment aids, keyboard shortcuts, and support for large-scale models.

These extensions are intended to evolve SCEditor-Web into a comprehensive smart contract modeling environment.

Table 3. Metaclasses defined in SCEditor-Web, grouped by diagram type and illustrated with their graphical icons.

Diagram Type	Metaclass	Definition/Role	SCEditor Notation
Structural	Struct	Defines a composite data type grouping multiple fields.	
Structural	Variable	Represents persistent state or storage elements in the contract.	
Structural	Function	Encapsulates executable logic, with parameters and return values.	
Structural	Enum	Declares symbolic constants for restricted value sets.	
Structural	Modifier	Specifies reusable preconditions for function execution.	
Structural	ErrorDeclaration	Declares structured error types for handling exceptional cases.	
Functional	Assignment	Defines value binding or state update operations.	
Functional	Call	Represents function or contract invocations with arguments.	
Functional	Condition	Models branching logic (e.g., if/else).	
Functional	Emit	Triggers events for off-chain listeners.	
Functional	Loop	Encodes iterative behavior (e.g., for, while).	
Structural/Functional	Literal	Represents constant values such as numbers or strings.	

5.2. Application Example: Remote Purchase

To demonstrate the capabilities of our editor, we selected a realistic use case: a smart contract implementing a Remote Purchase agreement. This type of contract models the interaction between a seller and a buyer in a trust-minimized way, where the funds are held until both parties confirm that the product or service has been delivered. This makes it a strong candidate for showcasing modeling complexity, role management, and LLM-based code synthesis.

The modeling process begins inside SCEditor-Web. The user starts by creating a contract component called “RemotePurchase” and defining global variables such as `seller`, `buyer`, `price`, and `state`. Using the editor’s main layer, the user defines the structural layout, including functions like `confirmPurchase`, `confirmReceived`, and `abort`. Each function node can be double-clicked to open the function layer, where the behavioral logic is built using visual blocks such as conditional statements, assignment operations, and emit statements. These elements are positioned interactively on the canvas and linked to represent control flow. For example, within the `confirmPurchase` function, the user may model a condition verifying the buyer’s identity, an assignment updating the state of the transaction, and the emission of an event to signal that the purchase has been confirmed. This layered and visual approach facilitates precise behavioral modeling without requiring manual code input, while maintaining conformance to the metamodel described in Table 3.

Once the contract model is finalized within the editor, it can be exported as a JSON file that is then wrapped into a prompt using our system’s template, which combines the model with a natural language instruction for the LLM. The resulting prompt is submitted to the configured LLM to generate the corresponding smart contract code.

In this case, we used the Gemini 2.5 Pro model to generate Solidity code based on the JSON file. The model successfully recognized and implemented the core components of the Remote Purchase contract. It created appropriate state variables, access checks (such as verifying `msg.sender` roles), and included event emissions. Functions such as `confirmPurchase` and `abort` were implemented with conditionals that corresponded closely to the modeled logic. One interesting observation is that the model introduced an additional state validation within the `confirmReceived` function, an element not explicitly defined in the original model, but which aligns with common best practices in contract safety. While this demonstrates the model’s capacity to infer sensible guard conditions, it also underscores the non-deterministic nature of LLM outputs, where generative reasoning may deviate from the source specification. This behavior highlights the importance of interpretability and post-generation validation when using LLMs in code-critical domains such as smart contract development.

We then analyzed the generated contract using our evaluation workflow. The Solidity file compiled successfully with no errors using `solc`. Slither analysis returned only minor informational warnings, with no critical vulnerabilities. The Semantic Fidelity Score for Gemini on this contract was 0.81, meaning that most logic blocks were correctly reflected in the output code. The contract also scored moderately on code quality, meeting standards of readability and alignment with best practices. These results suggest that Gemini was able to accurately capture the control flow, access control mechanisms, and behavioral transitions encoded in the structured JSON, although with occasional over-generalizations or redundant checks. While the generated code did not require post-correction to pass static analysis, future iterations of the pipeline may incorporate automatic post-processing to enforce stricter adherence to modeled behavior or domain-specific constraints.

These visual artifacts (Figure 4) help the reader to understand how the abstract model is mapped to executable code. While we have not yet tested this contract on a testnet or deployed it in a real-world application, this is planned for future work as part of our runtime operability experiments.

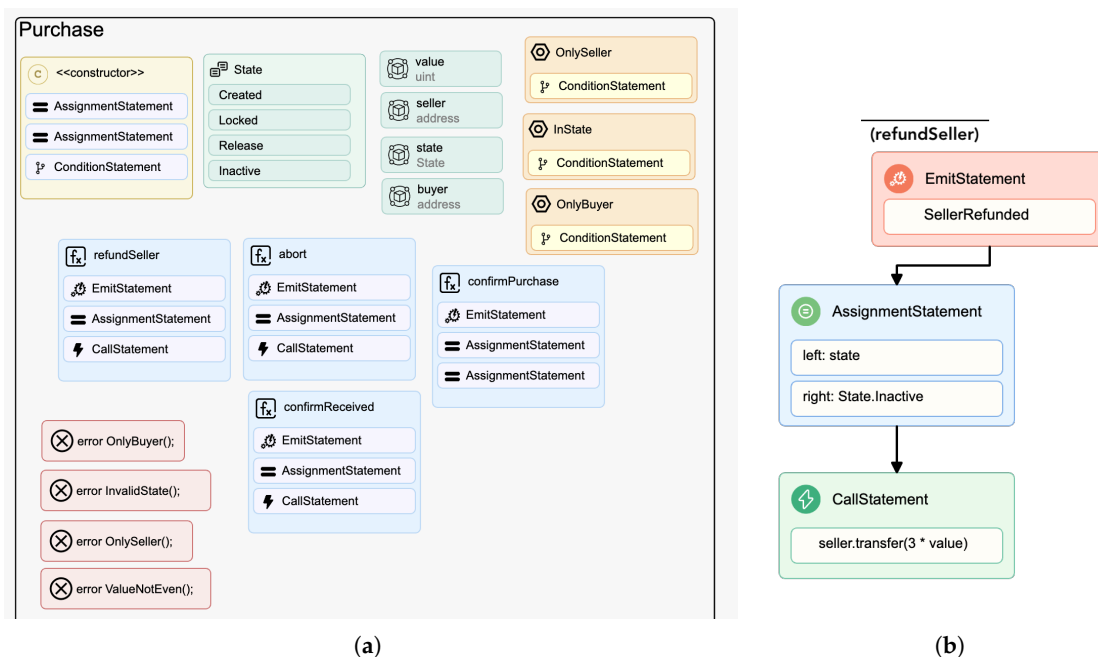


Figure 4. Remote Purchase case study modeled in SEditor-Web: (a) Structural diagram with constructor, structs, variables, functions, and error declarations. (b) Functional diagram of the `refundSeller` function, highlighting event emission, assignments, and calls.

5.3. Syntax Success Rate

The most fundamental requirement for any generated smart contract is syntactic validity. A contract that does not compile is unusable in any blockchain deployment scenario, regardless of its semantic or architectural structure. To measure this, we evaluated each model's output using the Solidity compiler (solc) and recorded whether the code compiled without errors.

Each of the four evaluated models (ChatGPT 4o, Claude 3.7 Sonnet, DeepSeek-V3 and Gemini 2.5 Pro) was prompted to generate Solidity code for three distinct contracts, resulting in a total of twelve contract files.

Surprisingly, all models achieved a **100% SSR**, indicating that every generated contract was syntactically valid and accepted by the compiler without error. This result suggests a significant advancement in the baseline Solidity generation capabilities of modern LLMs.

Figure 5 presents a comparative view of the SSR per model. While this figure may appear uniform due to perfect scores across the board, it plays an important role in establishing that syntax correctness appears to be largely solved, at least in our tested scenarios.

This finding permits subsequent evaluation stages (semantic fidelity and code quality) to focus entirely on deeper behavioral and structural correctness, knowing that the foundation (valid code) has already been satisfied.

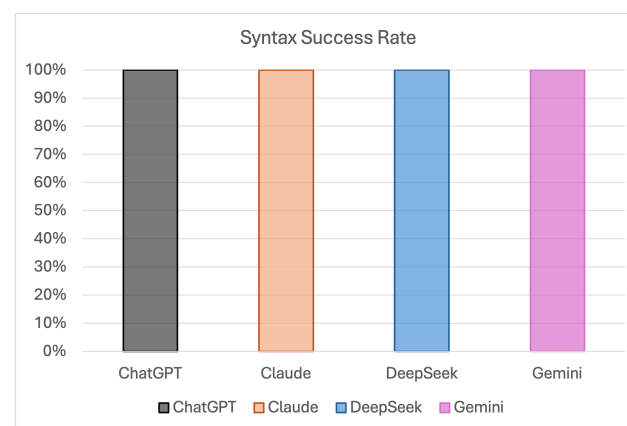


Figure 5. Syntax Success Rate (SSR) for all models, showing that each generated syntactically valid Solidity code across the three contracts.

5.4. Semantic Fidelity

While all models achieved perfect syntactic validity, this alone does not guarantee that the generated contracts captured the intended logic. To evaluate deeper semantic correctness, we computed the Semantic Fidelity Score as described in Section 4.

Table 4 reports the average SFS per model and contract. Higher scores reflect stronger alignment between the generated code and the original JSON specification. Results show that all models produced functionally recognizable contracts, but deviations emerged in more complex scenarios such as RemotePurchase, where multi-stage logic and state transitions were harder to reproduce.

Table 4. Average Semantic Fidelity Scores (0–5) for three benchmark contracts; higher scores indicate closer alignment between generated code and the source model.

Model	HotelInventory	BlindAuction	RemotePurchase
ChatGPT 4o	4.0	4.8	4.6
Claude 3.7	5.0	5.0	4.4
DeepSeek-V3	4.0	4.6	3.6
Gemini 2.5 Pro	5.0	5.0	4.6

Figure 6 illustrates the distribution of scores across the five evaluation criteria. Claude and Gemini exhibit strong, balanced performance across all axes, while ChatGPT performs reliably but shows some weaknesses in handling nested logic. DeepSeek displays greater variability, particularly in return behavior and complex control flows.

Overall, Gemini and Claude consistently produced code closely aligned with the modeled logic, particularly in contracts involving multiple functions or state transitions. ChatGPT performed well on simpler templates but struggled with deeper reasoning. DeepSeek, while promising, showed inconsistencies in reproducing more complex behaviors. These results highlight the uneven ability of LLMs to translate structured specifications into semantically faithful Solidity code.

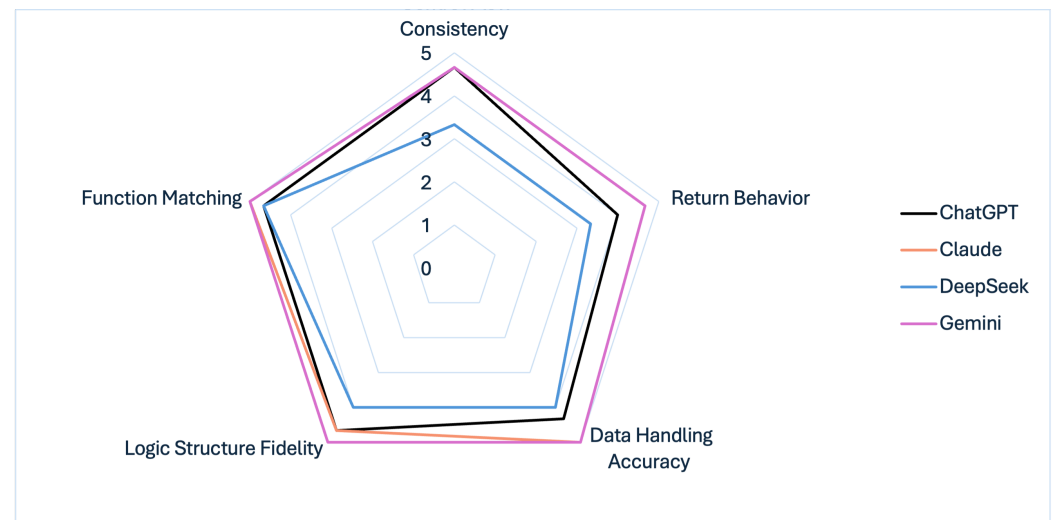


Figure 6. Radar plot of average Semantic Fidelity Scores across five dimensions: function matching, logic structure, data handling, return behavior, and control flow consistency.

5.5. Code Quality Evaluation

While syntactic and semantic correctness ensure functional validity, code that is poorly structured or difficult to maintain can still pose risks in real-world deployment. To capture these aspects, we applied the CQS defined in Section 4, which evaluates readability, modularity, naming quality, gas-awareness, and adherence to Solidity best practices.

Table 5 reports the average Code Quality Score per model and contract. Gemini and Claude generated contracts that consistently reflected clean modular design, well-named variables, and alignment with Solidity conventions. ChatGPT outputs were readable but occasionally relied on broader function scopes. DeepSeek, while competent, lagged slightly in modularity and naming.

Table 5. Code Quality Scores (0–5) per contract, based on readability, modularity, naming conventions, gas-awareness, and Solidity best practices.

Model	HotelInventory	BlindAuction	RemotePurchase
ChatGPT 4o	3.4	3.8	3.6
Claude 3.7	4.4	4.4	4.4
DeepSeek-V3	3.8	4.0	4.0
Gemini 2.5 Pro	4.4	4.4	4.4

Figure 7 complements the tabular view by aggregating average quality scores across all contracts. It illustrates the general consistency of Claude and Gemini, alongside greater variability in ChatGPT and DeepSeek outputs.

These results suggest that while most LLMs can generate syntactically valid smart contracts, producing code that is idiomatic, maintainable, and developer-friendly remains uneven. Claude and Gemini appear to better internalize Solidity design conventions, possibly due to higher-quality training corpora or more consistent decoding strategies. In contrast, ChatGPT occasionally sacrifices structural elegance for conciseness, and DeepSeek’s inconsistencies suggest room for improvement in understanding multi-layered logic or naming conventions. These patterns suggest that while LLMs can follow structure, only some begin to capture the deeper stylistic and semantic qualities that make generated contracts truly maintainable and trustworthy.

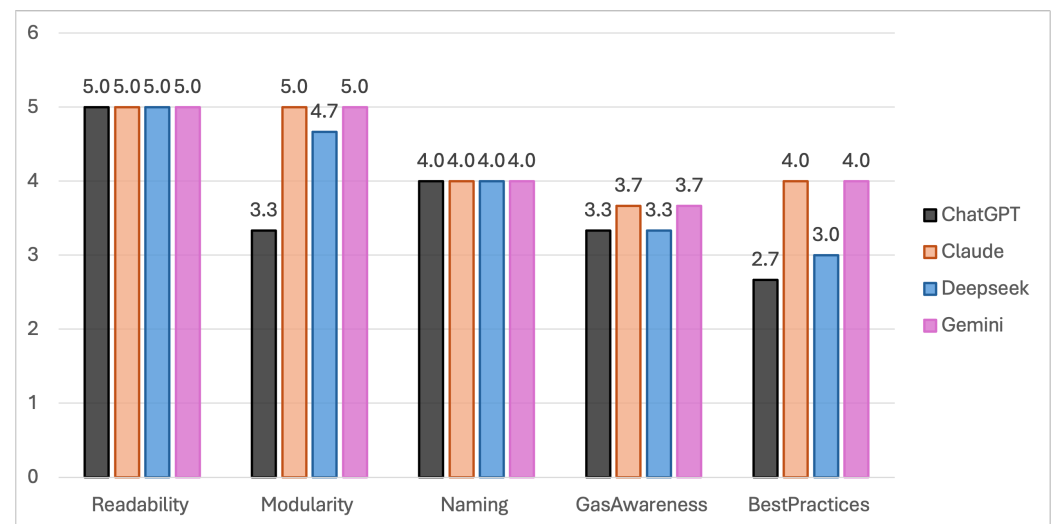


Figure 7. Average Code Quality Scores by model, evaluated using five heuristics: readability, modularity, naming, gas-awareness, and Solidity best practices.

5.6. Runtime Validation of Generated Contracts

While SSR, SFS, and CQS provide valuable indicators of model performance, they do not guarantee deployability or functional correctness on blockchain. To complement our static evaluation, we executed each generated contract within an Ethereum test environment (Hardhat v3), simulating the full lifecycle of interactions: contract initialisation, `confirmPurchase`, `confirmReceived`, and `refundSeller`.

Table 6 summarizes the runtime results. Three models (ChatGPT, Claude, and DeepSeek) successfully completed the entire workflow, correctly transitioning through contract states and handling ether transfers. In contrast, Gemini encountered two critical runtime failures. First, it rejected valid calls to `confirmPurchase` due to mismatched payment validation rules. Second, it triggered a `Transfer failed` runtime error in `refundSeller`.

These failures occurred despite Gemini achieving the highest composite score in static evaluation, highlighting a significant disconnect between static indicators and actual runtime behavior. This underscores the importance of validating not only structural and semantic correctness, but also executable behavior under realistic conditions. As shown in Table 6, even minor inconsistencies in fund handling logic can render a smart contract non-functional—despite appearing valid on inspection.

These findings highlight a critical insight: syntactic validity and even strong semantic alignment do not ensure runtime operability. Gemini’s failures illustrate how subtle deviations in value handling and fund transfer mechanisms can render a contract unusable, despite appearing correct under static inspection.

Table 6. Runtime validation of the Remote Purchase contract. A checkmark (✓) indicates successful execution of the operation, while a cross (x) indicates failure.

Model	Initialization	confirmPurchase	confirmReceived	refundSeller
ChatGPT 4o	✓	✓	✓	✓
Claude 3.7 Sonnet	✓	✓	✓	✓
DeepSeek-V3	✓	✓	✓	✓
Gemini 2.5 pro	✓	✓	✓	x

5.7. Normalized Composite Score

To offer a unified performance indicator that incorporates syntax, semantic fidelity, and code quality, we computed the Normalized Composite Score for each model. This score aggregates the three core metrics: SSR, SFS, and CQS by normalizing each to a 0–1 scale and taking their average.

Table 7 presents the aggregated NCS values for each model. Given the uniform 100% SSR across all evaluated contracts, differences in NCS primarily reflect performance variations in semantic alignment and code quality.

Table 7. Normalized Composite Scores (NCSs) per model, aggregating Syntax Success Rate (SSR), Semantic Fidelity Score (SFS), and Code Quality Score (CQS).

Model	SSR	Avg. SFS	Avg. CQS	NCS
ChatGPT 4o	100%	4.47	3.60	0.871
Claude 3.7	100%	4.73	4.40	0.942
DeepSeek-V3	100%	3.87	3.93	0.853
Gemini 2.5 Pro	100%	4.87	4.40	0.951

Figure 8 provides a radar visualization of each model’s normalized metrics across the three evaluation dimensions. Gemini slightly outperforms Claude in overall balance, while ChatGPT and DeepSeek show lower average fidelity and quality scores.

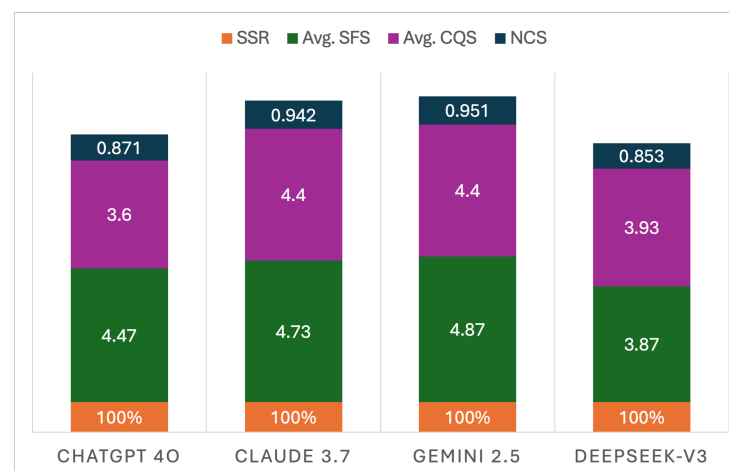


Figure 8. Composite radar plot showing normalized SSR, SFS, and CQS for each model.

These NCS results reinforce earlier findings: Gemini and Claude consistently outperform the other models in both fidelity and structural soundness, ChatGPT performed reliably with consistent outputs, though its code quality was comparatively lower, while DeepSeek reliably captured core structures and basic logic patterns but showed some limitations in preserving complex logic and adhering to idiomatic Solidity conventions. The

high NCS of Gemini (0.951) suggests not only syntactic fluency but also effective reasoning about control flows and data constraints.

6. Discussion

SCEditor-Web addresses the key challenges in existing smart contract development approaches, including the steep learning curve of blockchain development, the error-prone nature of manual coding, the absence of standardized modeling and validation tools, platform specificity, and difficulties related to gas cost optimization. By integrating MDE-driven visual representation with LLM-based code generation, our approach combines the strengths of existing methods while mitigating many of their limitations.

In the following discussion, we first position our approach with respect to related work, highlighting where SCEditor-Web advances beyond existing methodologies. We then outline the strengths and limitations of the study to provide a balanced and critical evaluation.

6.1. Positioning of Our Approach vs. Related Work

In this section, we position SCEditor-Web within the broader landscape of smart contract development. Unlike the related work (Section 2), which primarily reviewed existing approaches, here we highlight how our solution compares to representative tools and methodologies, including both model-driven and AI-assisted techniques. We also contrast SCEditor-Web with our earlier prototype, SCEditor, to illustrate its progression toward a more flexible, AI-powered, and platform-independent environment.

Compared to MDE-based tools such as [16,25], our approach avoids tight coupling with Ethereum by adopting a blockchain-independent metamodel, enabling broader applicability across blockchain platforms. It builds upon our earlier work, SCEditor [24], a prototype graphical editor based on Eclipse Sirius, and extends it into a fully web-based environment. While the original SCEditor facilitated visual modeling, it was constrained by its IDE-bound architecture and lacked smart contract code generation capabilities. SCEditor-Web overcomes these limitations by integrating LLM-based code generation and multi-platform deployment, thereby broadening applicability and improving usability.

Although solutions such as MUISCA [26] and iContractML 2.0 [30] demonstrate promising cross-platform capabilities, they rely on static transformation rules, which limits their flexibility. In contrast, SCEditor-Web introduces a JSON-based intermediate representation that leverages the generative flexibility of LLMs, enabling AI-assisted evolution of smart contract code beyond rigid rule based systems. Unlike DSL-based systems such as SmaC [28] or domain-specific graphical tools like the one proposed by [29], SCEditor-Web offers a general-purpose visual language tailored to smart contracts, without being constrained by a specific industry or execution context. In addition, its graphical modeling interface bridges the abstraction gap often present in textual DSLs, improving accessibility for non-programmers while preserving semantic rigor.

Structured AI approaches such as [35] highlight the potential of LLMs to transform domain models into smart contracts. However, they remain constrained by modeling rigidities (e.g., BPMN expressiveness or static annotations), which often lack constructs specific to decentralized logic and smart contract semantics. SCEditor-Web addresses these limitations through a platform-independent metamodel for smart contract logic, serialized into LLM-friendly formats, and enabling generation across Solidity, Vyper, and other programming languages.

In contrast to natural language-driven approaches like Chat2Code [33] and AIASCG [34], which prioritize accessibility but struggle with complex logic, our visual modeling layer enables precise control over contract semantics. By grounding the prompt generation in a

structured metamodel, we reduce ambiguity and enhance the accuracy and consistency of the generated code.

Unlike auditing-focused solutions such as SmartLLM [36] or QuadraCode [39], which only operate on post-written code, our workflow supports design-to-code traceability, aiding both development and verification.

Furthermore, while lifecycle frameworks such as the one proposed by Mohan et al. [38] support end-to-end automation, their scope remains restricted to Solidity. SCEditor-Web's architecture, by contrast, is designed for platform independence, laying the groundwork for future extensibility across multiple blockchain ecosystems.

6.2. Strengths and Limitations of the Study

Building upon these efforts in model-driven engineering and AI-assisted development, our approach advances the state of the art by combining domain-specific graphical modeling tailored explicitly for smart contracts, and LLM-based code generation. In sum, Figure 9 highlights the challenges addressed by SCEditor-Web and the corresponding solutions. The editor combines a platform-independent visual modeling environment, grounded in a formal metamodel, to streamline smart contract design and reduce complexity. In parallel, it integrates prompt-based LLM generation to automate code generation across multiple blockchain platforms. Additionally, SCEditor-Web incorporates AI-driven gas cost optimization, supporting the generation of contracts that are not only correct and portable but also cost-efficient. This hybrid approach reduces manual effort, ensures better alignment with user-defined models, and supports the development of smart contracts with minimal technical debt. By improving automation, maintainability, and portability, SCEditor-Web contributes to more accessible, reliable, and efficient smart contract engineering.

Runtime validation revealed that syntactic and semantic correctness do not guarantee deployability. While most models executed the purchase flow correctly, Gemini failed at critical points due to mismatched payment validation and transfer mechanisms. These runtime failures are particularly instructive: they demonstrate how high static scores can mask hidden operational flaws, and they highlight that even advanced LLMs can generate code that aligns with specifications on paper but fails under realistic blockchain execution. For broader LLM adoption in smart contract engineering, this suggests that static checks alone are insufficient, developers must integrate runtime simulation, stress testing, and conformance to platform-specific rules into their workflows. Without such safeguards, LLM-generated contracts risk appearing reliable yet breaking under real transaction flows, which could have severe consequences in production contexts.

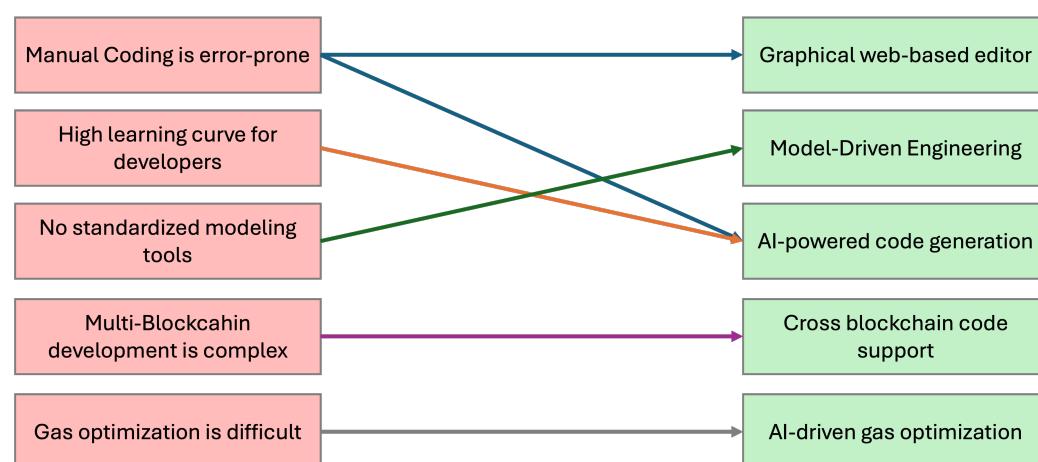


Figure 9. Key challenges in smart contract development and the corresponding solutions proposed by SCEditor-Web.

Concluding this section, it should be noted that despite the potential of our editor in integrating model-driven inputs with LLM-generated smart contracts, several challenges remain to be addressed:

- **Language Scope:** All contract generations and analyses were restricted to Solidity, the dominant smart contract language for the Ethereum Virtual Machine (EVM). Although our metamodel is designed to be independent of any blockchain, and the editor can accommodate structures compatible with other platforms (e.g., Solana or Polkadot), the evaluation focused solely on Solidity to ensure metric consistency and simplify tooling integration. Future work may include testing Rust (for Solana) or Vyper (EVM-compatible) to assess cross-chain adaptability.
- **Security Validation:** While our evaluation framework covered syntax, semantic fidelity, code quality, and runtime execution, it did not include dedicated security verification. Key aspects such as vulnerability scanning, formal verification, and conformance to blockchain-specific operational rules (e.g., gas metering, access control, reentrancy resistance) were not yet addressed. Future work should integrate our workflow with established auditing tools and formal methods to ensure that generated contracts are not only functional but also secure.
- **Zero-Shot Prompting:** The models were evaluated in a zero-shot configuration. We did not explore performance under few-shot prompts, chain-of-thought scaffolding, or system prompt customization. While this choice allowed for a clean comparison of each model's default reasoning capabilities, it also may under-represent the full potential of each LLM under guided prompting scenarios.
- **Human-Dependent Evaluation:** While the semantic fidelity and code quality scoring rubrics were carefully defined and applied consistently, they still involve manual interpretation. Inter-rater reliability was not measured, and results may reflect the evaluator's familiarity with Solidity best practices and metamodel constraints. Incorporating multi-reviewer scoring or automated fidelity checks could enhance the reproducibility of this component.
- **LLM Reliability:** While the editor leverages LLMs for code synthesis, these models remain probabilistic and may occasionally produce hallucinations, incomplete logic, or semantic drift from the source model. Our multi-metric evaluation (syntax, semantic fidelity, runtime validation) helps detect such cases, but full determinism and logical soundness remain open challenges for future work.

7. Conclusions and Future Work

This paper presented SCEditor-Web, a web-based modeling environment that facilitates the design and development of smart contracts by combining MDE with Gen-AI techniques. The platform enables users to visually model both the structural and behavioral aspects of smart contracts, serialize these models into a standardized JSON format, and generate platform-specific code via prompt-based interactions with LLMs, all without requiring manual programming.

Our evaluation with four state-of-the-art LLMs (ChatGPT-4o, Claude 3.7 Sonnet, DeepSeek-V3, and Gemini 2.5 Pro) demonstrated strong performance in syntactic correctness, semantic fidelity, and code quality. Solidity-based case studies, including the Remote Purchase contract, validated the practicality of the visual-to-code pipeline. Crucially, runtime validation on a testnet revealed critical differences between models: while ChatGPT, Claude, and DeepSeek executed the complete lifecycle successfully, Gemini failed due to stricter payment logic and transfer errors. These results emphasize that static correctness alone is insufficient and that runtime validation is essential for assessing deployability.

Overall, SCEditor-Web shows that integrating MDE with LLM-driven code generation can reduce technical barriers and automate key aspects of smart contract development. However, the current scope does not yet address formal security validation or platform-specific execution constraints.

Future Work

Planned enhancements will focus on expanding both the modeling expressiveness and the intelligence of the code generation process. On the editor side, we aim to extend behavioral modeling support, incorporate real-time model validation, and improve usability through features such as undo/redo, multiobject manipulation, and collaborative modeling. Enhancing accessibility for diverse user groups is also under consideration.

On the AI integration front, we plan to investigate advanced prompting strategies (few-shot, multi-turn, and prompt chaining) to improve semantic alignment. Future evaluations will incorporate automated unit testing, formal verification, and security auditing to better assess robustness against known vulnerabilities. Additionally, we intend to broaden language support, particularly Rust (for Solana) and Ink! (for Polkadot), to validate the generality of the metamodel beyond Solidity.

In the long term, our goal is to unify modeling, generation, runtime deployment, and testing within a single, seamless environment. By bridging formal modeling and generative AI, SCEditor-Web represents a step toward more accessible, reliable, and platform-independent smart contract engineering.

Author Contributions: Conceptualization, Y.A.H. and N.L.; methodology, Y.A.H.; software, Y.A.H.; validation, N.L.; formal analysis, Y.A.H.; investigation, Y.A.H. and N.L.; resources, Y.A.H.; data curation, Y.A.H. and N.L.; writing—original draft preparation, Y.A.H. and N.L.; writing—review and editing, Y.A.H., N.L. and S.M.; visualization, Y.A.H.; supervision, S.M.; project administration, N.L. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The original data presented in this study are openly available in GitHub at <https://github.com/YassineDev91/smart-contract-eval> (accessed on 23 September 2025).

Acknowledgments: During the preparation of this manuscript, the author used OpenAI's ChatGPT (GPT-5, August 2025 release) to assist in refining the language of several sections and to improve readability. All outputs were reviewed, edited, and validated by the author, who takes full responsibility for the final content of this publication.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Taherdoost, H. Smart contracts in blockchain technology: A critical review. *Information* **2023**, *14*, 117. [CrossRef]
2. Lai, J.; Yan, K. VortexDraft: A blockchain smart contract auto-generation system based on named entity recognition. In Proceedings of the IET Conference Proceedings CP989, Sanya, China, 1–4 August 2024; Volume 2024, pp. 350–356.
3. Gan, J.; Su, J.; Lin, K.; Zheng, Z. FinanceFuzz: Fuzzing Smart Contracts with Financial Properties. *Blockchain Res. Appl.* **2025**, 100301. [CrossRef]
4. Guo, L. EXPRESS: Smart Contracts in Supply Chains. *J. Mark. Res.* **2025**, *62*, 00222437251314003. [CrossRef]
5. Bawa, G.; Singh, H.; Rani, S.; Kataria, A.; Min, H. Exploring perspectives of blockchain technology and traditional centralized technology in organ donation management: A comprehensive review. *Information* **2024**, *15*, 703. [CrossRef]
6. Mars, R.; Cheikhrouhou, S.; Kallel, S.; Hadj Kacem, A. A survey on automation approaches of smart contract generation. *J. Supercomput.* **2023**, *79*, 16065–16097. [CrossRef]

7. Kannengiesser, N.; Lins, S.; Sander, C.; Winter, K.; Frey, H.; Sunyaev, A. Challenges and common solutions in smart contract development. *IEEE Trans. Softw. Eng.* **2021**, *48*, 4291–4318. [\[CrossRef\]](#)
8. Curty, S.; Härer, F.; Fill, H.G. Blockchain application development using model-driven engineering and low-code platforms: A survey. In Proceedings of the International Conference on Business Process Modeling, Development and Support, Leuven, Belgium, 6–7 June 2022; Springer: Berlin/Heidelberg, Germany, 2022; pp. 205–220.
9. Coblenz, M.; Sunshine, J.; Aldrich, J.; Myers, B.A. Smarter smart contract development tools. In Proceedings of the 2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB), Montreal, QC, Canada, 27 May 2019; pp. 48–51.
10. Zafar, A.; Azam, F.; Latif, A.; Anwar, M.W.; Safdar, A. Exploring the Effectiveness and Trends of Domain-Specific Model Driven Engineering: A Systematic Literature Review (SLR). *IEEE Access* **2024**, *12*, 86809–86830. [\[CrossRef\]](#)
11. Dorado, J.P.C.; Dulce-Villarreal, E.; Hurtado, J.A. Model Driven Engineering Tool for the Generation of Interoperable Smart Contracts. In Proceedings of the Colombian Conference on Computing, Manizales, Colombia, 4–6 September 2024; Springer: Berlin/Heidelberg, Germany, 2024; pp. 324–331.
12. Samanipour, A.; Bushehrian, O.; Robles, G. MDAPW3: MDA-based development of blockchain-enabled decentralized applications. *Sci. Comput. Program.* **2025**, *239*, 103185. [\[CrossRef\]](#)
13. Ait Hsain, Y.; Laaz, N.; Mbarki, S. A Common Metamodel for Smart Contracts Development. In Proceedings of the International Conference on Advanced Intelligent Systems for Sustainable Development, Agadir, Morocco, 17–23 December 2024; Springer: Berlin/Heidelberg, Germany, 2025; pp. 358–365.
14. Curty, S.; Härer, F.; Fill, H.G. Design of blockchain-based applications using model-driven engineering and low-code/no-code platforms: A structured literature review. *Softw. Syst. Model.* **2023**, *22*, 1857–1895. [\[CrossRef\]](#)
15. Köpke, J.; Meroni, G.; Salnitri, M. Designing secure business processes for blockchains with SecBPMN2BC. *Future Gener. Comput. Syst.* **2023**, *141*, 382–398. [\[CrossRef\]](#)
16. Jurgelaitis, M.; ėponienė, L.; Butkienė, R. Solidity code generation from UML state machines in model-driven smart contract development. *IEEE Access* **2022**, *10*, 33465–33481. [\[CrossRef\]](#)
17. Hsain, Y.A.; Laaz, N.; Mbarki, S. Ethereum’s smart contracts construction and development using model driven engineering technologies: A review. *Procedia Comput. Sci.* **2021**, *184*, 785–790. [\[CrossRef\]](#)
18. Nassar, E.Y.; Mazen, S.; Craß, S.; Helal, I.M. Modelling blockchain-based systems using model-driven engineering. In Proceedings of the 2023 Fifth International Conference on Blockchain Computing and Applications (BCCA), Kuwait, Kuwait, 24–26 October 2023; pp. 329–334.
19. Khalid, S.; Brown, C. Evaluating Capabilities and Perspectives of Generative AI Tools in Smart Contract Development. In Proceedings of the 7th ACM International Symposium on Blockchain and Secure Critical Infrastructure, Meliá Hanoi Hanoi, Vietnam, 25–29 August 2025; pp. 1–12.
20. Napoli, E.A.; Barbàra, F.; Gatteschi, V.; Schifanella, C. Leveraging large language models for automatic smart contract generation. In Proceedings of the 2024 IEEE 48th Annual Computers Software, and Applications Conference (COMPSAC), Osaka, Japan, 2–4 July 2024; pp. 701–710.
21. Barbàra, F.; Napoli, E.A.; Gatteschi, V.; Schifanella, C. Automatic smart contract generation through llms: When the stochastic parrot fails. In Proceedings of the 6th Distributed Ledger Technology Workshop, Turin, Italy, 14–15 May 2024.
22. Ding, H.; Liu, Y.; Piao, X.; Song, H.; Ji, Z. SmartGuard: An LLM-enhanced framework for smart contract vulnerability detection. *Expert Syst. Appl.* **2025**, *269*, 126479. [\[CrossRef\]](#)
23. Busch, D.; Balczyk, A.; Smyth, S.; Steffen, B. LLM-based code generation and system migration in language-driven engineering. *Int. J. Softw. Tools Technol. Transf.* **2025**, *27*, 137–147. [\[CrossRef\]](#)
24. Hsain, Y.A.; Laaz, N.; Mbarki, S. SEditor: A Graphical Editor Prototype for Smart Contract Design and Development. *Int. J. Adv. Comput. Sci. Appl.* **2024**, *15*, 1185–1195. [\[CrossRef\]](#)
25. Velasco, G.; Vaz, N.A.; Carvalho, S.T. A High-Level Metamodel for Developing Smart Contracts on the Ethereum Virtual Machine. In Proceedings of the Workshop em Blockchain: Teoria, Tecnologias e Aplicações (WBlockchain), Porto Alegre, Brazil, 24 June 2024; SBC: Porto Alegre, Brazil, 2024; pp. 97–110.
26. Dulce-Villarreal, E.; Hernandez, G.; Insuasti, J.; Hurtado, J.; Garcia-Alonso, J. Validation of MUISCA: A MDE-Based Tool for Interoperability of Healthcare Environments Using Smart Contracts in Blockchain. In *Envisioning the Future of Health Informatics and Digital Health*; IOS Press: Amsterdam, The Netherlands, 2025; pp. 255–259.
27. Wöhrer, M.; Zdun, U. Domain specific language for smart contract development. In Proceedings of the 2020 IEEE International Conference on Blockchain and Cryptocurrency (ICBC), Toronto, ON, Canada, 2–6 May 2020; pp. 1–9.
28. Gómez Macías, C. SmaC: A Model-Based Framework for the Development of Smart Contracts. Ph.D. Thesis, Universidad Rey Juan Carlos, Móstoles, Spain, 2023.

29. Ye, X.; Zeng, N.; Tao, X.; Han, D.; König, M. Smart contract generation and visualization for construction business process collaboration and automation: Upgraded workflow engine. *J. Comput. Civ. Eng.* **2024**, *38*, 04024030. [\[CrossRef\]](#)
30. Hamdaq, M.; Met, L.A.P.; Qasse, I. iContractML 2.0: A domain-specific language for modeling and deploying smart contracts onto multiple blockchain platforms. *Inf. Softw. Technol.* **2022**, *144*, 106762. [\[CrossRef\]](#)
31. Alzhrani, F.; Saeedi, K.; Zhao, L. A Business Process Modeling Pattern Language for Blockchain Application Requirement Analysis. In *Understanding Blockchain Applications from Architectural and Business Process Perspectives*; The University of Manchester: Manchester, UK, 2022.
32. Daspe, E.; Durand, M.; Hatin, J.; Bradai, S. Benchmarking Large Language Models for Ethereum Smart Contract Development. In Proceedings of the 2024 6th Conference on Blockchain Research & Applications for Innovative Networks and Services (BRAINS), Berlin, Germany, 9–11 October 2024; pp. 1–4.
33. Qasse, I.; Mishra, S.; Hamdaq, M. Chat2Code: Towards conversational concrete syntax for model specification and code generation, the case of smart contracts. *arXiv* **2021**, arXiv:2112.11101. [\[CrossRef\]](#)
34. Tong, Y.; Tan, W.; Guo, J.; Shen, B.; Qin, P.; Zhuo, S. Smart contract generation assisted by AI-based word segmentation. *Appl. Sci.* **2022**, *12*, 4773. [\[CrossRef\]](#)
35. Gao, S.; Liu, W.; Zhu, J.; Dong, X.; Dong, J. BPMN-LLM: Transforming BPMN Models into Smart Contracts Using Large Language Models. *IEEE Softw.* **2025**, *42*, 50–57. [\[CrossRef\]](#)
36. Kevin, J.; Yugopuspito, P. SmartLLM: Smart Contract Auditing using Custom Generative AI. *arXiv* **2025**, arXiv:2502.13167. [\[CrossRef\]](#)
37. Krichen, M. Strengthening the security of smart contracts through the power of artificial intelligence. *Computers* **2023**, *12*, 107. [\[CrossRef\]](#)
38. Mohan, M.S.; Swamy, T.; Reddy, V.C. Strengthening Smart Contracts: An AI-Driven Security Exploration. *Glob. J. Comput. Sci. Technol.* **2023**, *23*, 57–67.
39. Upadhyay, J.; Upadhyay, K.; Sainju, A.; Poudel, S.; Hasan, M.N.; Poudel, K.; Ranganathan, J. QuadraCode AI: Smart Contract Vulnerability Detection with Multimodal Representation. In Proceedings of the 2024 33rd International Conference on Computer Communications and Networks (ICCCN), Kailua-Kona, HI, USA, 29–31 July 2024; pp. 1–9.
40. Sun, J.; Long, H.W.; Kang, H.; Fang, Z.; El Saddik, A.; Cai, W. A Multidimensional Contract Design for Smart Contract-as-a-Service. *IEEE Trans. Comput. Soc. Syst.* **2025**. [\[CrossRef\]](#)
41. Brambilla, M.; Cabot, J.; Wimmer, M. *Model-Driven Software Engineering in Practice*; Morgan & Claypool Publishers: San Rafael, CA, USA, 2017.
42. Viyović, V.; Maksimović, M.; Perišić, B. Sirius: A rapid development of DSM graphical editor. In Proceedings of the IEEE 18th International Conference on Intelligent Engineering Systems INES 2014, Tihany, Hungary, 3–5 July 2014; pp. 233–238.
43. Abdelmalek, H.; Khriiss, I.; Jakimi, A. Towards an effective approach for composition of model transformations. *Front. Comput. Sci.* **2024**, *6*, 1357845. [\[CrossRef\]](#)
44. Chen, M.; Tworek, J.; Jun, H.; Yuan, Q.; Pinto, H.P.D.O.; Kaplan, J.; Edwards, H.; Burda, Y.; Joseph, N.; Brockman, G.; et al. Evaluating large language models trained on code. *arXiv* **2021**, arXiv:2107.03374. [\[CrossRef\]](#)
45. Le, H.; Chen, H.; Saha, A.; Gokul, A.; Sahoo, D.; Joty, S. Codechain: Towards modular code generation through chain of self-revisions with representative sub-modules. *arXiv* **2023**, arXiv:2310.08992.
46. Achiam, J.; Adler, S.; Agarwal, S.; Ahmad, L.; Akkaya, I.; Aleman, F.L.; Almeida, D.; Altenschmidt, J.; Altman, S.; Anadkat, S.; et al. Gpt-4 technical report. *arXiv* **2023**, arXiv:2303.08774. [\[CrossRef\]](#)
47. Anthropic. Claude. Artificial Intelligence Model. 2023. Available online: <https://www.anthropic.com> (accessed on 5 April 2025).
48. Liu, A.; Feng, B.; Xue, B.; Wang, B.; Wu, B.; Lu, C.; Zhao, C.; Deng, C.; Zhang, C.; Ruan, C.; et al. Deepseek-v3 technical report. *arXiv* **2024**, arXiv:2412.19437.
49. DeepMind, G. Gemini 2.5 Pro Preview: Even Better Coding Performance. 2025. Available online: <https://deepmind.google/technologies/gemini/pro/> (accessed on 8 May 2025).
50. Sobo, A.; Mubarak, A.; Baimagambetov, A.; Polatidis, N. Evaluating LLMs for code generation in HRI: A comparative study of ChatGPT, gemini, and claude. *Appl. Artif. Intell.* **2025**, *39*, 2439610. [\[CrossRef\]](#)
51. Zhang, K.; Wang, D.; Xia, J.; Wang, W.Y.; Li, L. Algo: Synthesizing algorithmic programs with generated oracle verifiers. *Adv. Neural Inf. Process. Syst.* **2023**, *36*, 54769–54784.
52. Reynolds, L.; McDonnell, K. Prompt programming for large language models: Beyond the few-shot paradigm. In Proceedings of the Extended Abstracts of the 2021 CHI Conference on Human Factors in Computing Systems, Yokohama, Japan, 8–13 May 2021; pp. 1–7.
53. Schulhoff, S.; Ilie, M.; Balepur, N.; Kahadze, K.; Liu, A.; Si, C.; Li, Y.; Gupta, A.; Han, H.; Schulhoff, S.; et al. The prompt report: A systematic survey of prompting techniques. *arXiv* **2024**, arXiv:2406.06608.

-
54. Jiang, J.; Wang, F.; Shen, J.; Kim, S.; Kim, S. A survey on large language models for code generation. *arXiv* **2024**, arXiv:2406.00515. [[CrossRef](#)]
 55. Kojima, T.; Gu, S.S.; Reid, M.; Matsuo, Y.; Iwasawa, Y. Large language models are zero-shot reasoners. *Adv. Neural Inf. Process. Syst.* **2022**, *35*, 22199–22213.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.